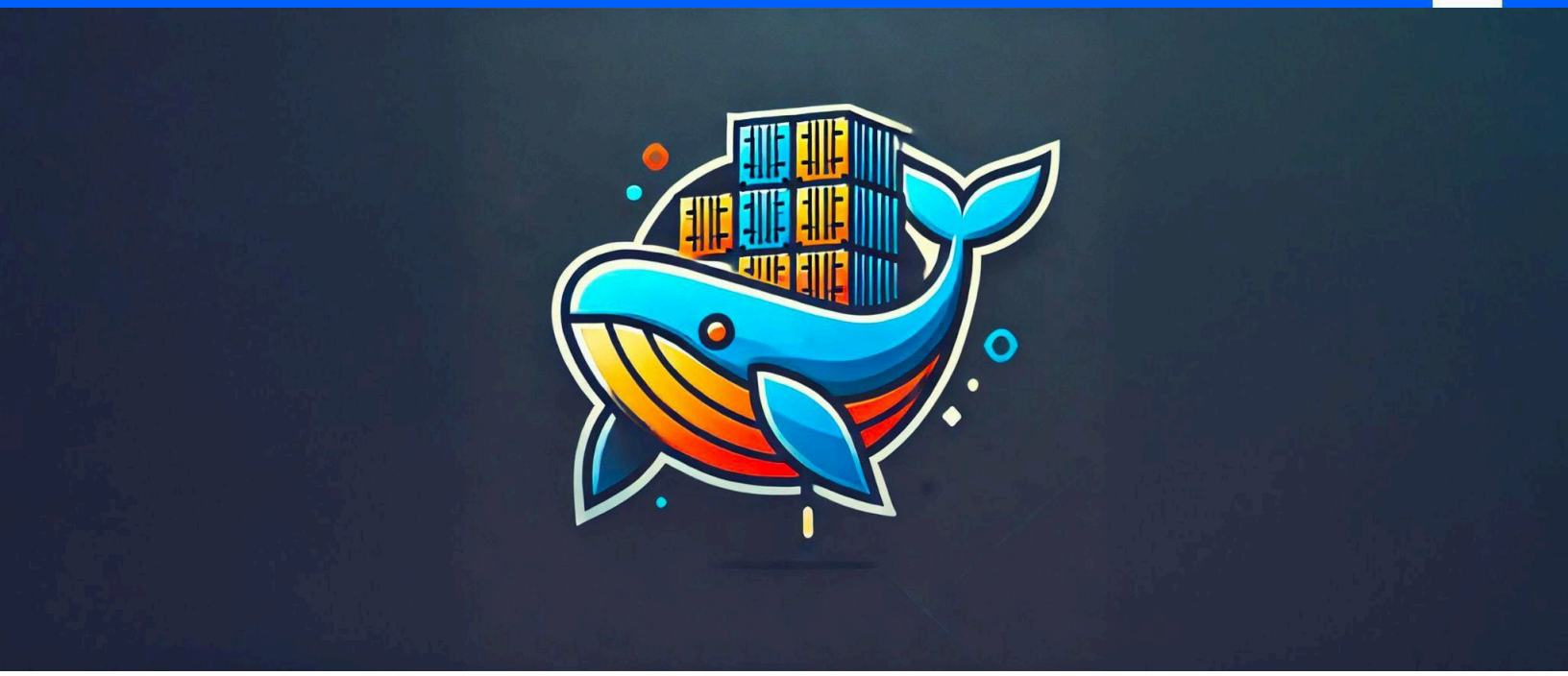
# Docker

Embarquez pour un voyage dans l'univers Docker où même les conteneurs ont le sens de l'humour !



## Introduction du sujet

Prêts à découvrir comment ces petites boîtes virtuelles rendent le déploiement d'applications aussi simple que de dire 'container' ?"

Docker est un outil qui permet de créer des "conteneurs" isolés contenant tout ce dont une application a besoin pour fonctionner. Ces conteneurs sont portables et peuvent être utilisés dans divers domaines :

- **Développement d'applications :** fournit un environnement de développement cohérent et portable.
- **Déploiement d'applications :** facilite le déploiement sur différentes plateformes sans se soucier des différences d'environnement.

- **Microservices :** Aide à la gestion et à l'évolutivité des services en les isolant dans des conteneurs distincts.
- **Tests et CI/CD :** Utilisé pour exécuter des tests automatisés et pour implémenter des pipelines CI/CD, assurant une intégration continue et une livraison continue.

## Job 01

Créer une VM Debian en **mode console** 8 Go DD / 1 Go RAM / 1 vCPU , installer docker (cli seulement).

## Job 02

Tester l'installation de docker avec le conteneur « hello-world » et se familiariser avec **les commandes.**

## Job 03

Utilisation de **« Dockerfile »** pour recréer le conteneur « helloworld » depuis une image Debian minimum.

## Job 04

Utilisation de **Dockerfile** pour créer une image **SSH** (compte : root et mot de passe : root123) sans utiliser une **image SSH existante**, voir redirection des ports (utiliser un autre port que le 22) , créé et lancez le conteneur et se connecter pour vérifier l'accès SSH.

## Job 05

Tout informaticien étant « flemmard » , il faut faire des **alias** pour les **commandes** docker en **CLI** , à mettre dans **~/.bashrc** pour manipuler les images / containers.

## Job 06

Se renseigner sur l'utilisation de volumes entre deux conteneurs et la gestion des volumes.

## Job 07

À l'aide d'un fichier **yml** , de **docker-compose** faire deux conteneurs : **nginx** et **FTP** liés entre eux. Création d'un volume commun pour accéder au dossier web.
Créer sur **votre PC** un fichier **index.html**, dans ce fichier faites afficher votre nom/prénom). Installer **FileZilla** sur votre PC , se connecter en FTP sur le **conteneur FTP** pour envoyer le fichier index.html, et regarder le résultat.

## Job 08

**Sans utiliser une image nginx existante** , utilisation de **« Dockerfile »** pour créer un conteneur nginx, voir redirection des ports, et se connecter.

## Job 09

Création et utilisation d'un **"registry" local.** Et ajouter une **« UI »** pour le gérer depuis une interface web.

## Job 10

Faire un script bash , pour **effacer totalement Docker** (les images , volumes , conteneurs, et paquets correspondants) et rendre le système propre.

Et aussi un script pour **automatiser l'installation de docker** sur une machine Debian.

## Job 11

Découverte de **portainer**.
Refaire les **Job 2 à 9** , en utilisant **l'interface graphique** de portainer. Se renseigner sur les alternatives à Portainer.

## Pour aller plus loin

On complique un peu le système, il va falloir faire **l'équivalent de XAMPP**
Avec un **fichier Dockerfile** et un fichier yml faire :

- ➔ un serveur **nginx** (ou **apache**) avec PHP
- ➔ un serveur **MariaDB/MySQL** (avec phpMyAdmin)
- ➔ un serveur **FTP**
- ➔ un **volume** pour stocker l'ensemble des données **communes aux différents serveurs**

  Tester le système.

# Compétences visées

- ➔ Administrer et sécuriser les infrastructures virtualisées
- ➔ Mettre en production des évolutions de l'infrastructure

# Rendu

Le projet est à rendre sur: **https://github.com/prenom-nom/docker**, un fichier Doc par Job avec les commandes , scripts utilisés. Pensez à mettre votre repository en **public**.
L'évaluation se fera sous forme de présentation sans support à l'équipe pédagogique.

# Base de connaissances

- ➔ Documentation Docker
- ➔ Forum Docker Community
- ➔ Docker Captain's Blog
- ➔ Portainer
- ➔ Docker Volumes